

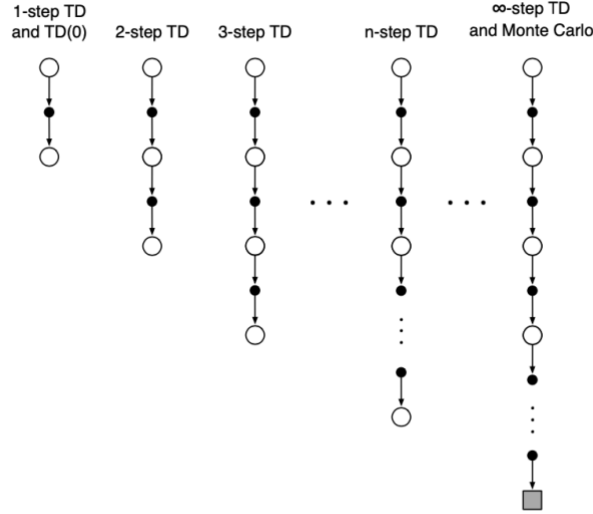


Figure 7.1: The backup diagrams of n -step methods. These methods form a spectrum ranging from one-step TD methods to Monte Carlo methods.

n -step TD for estimating $V \approx v_\pi$

Input: a policy π

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take an action according to $\pi(\cdot | S_t)$

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then $T \leftarrow t + 1$

$\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ ($G_{\tau:\tau+n}$)

$V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$

 Until $\tau = T - 1$

n -step Sarsa for estimating $Q \approx q_*$ or q_π

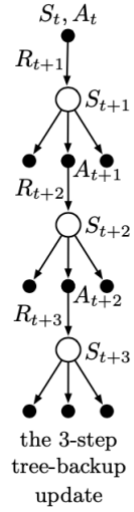
Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε -greedy with respect to Q , or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:
 Initialize and store $S_0 \neq \text{terminal}$
 Select and store an action $A_0 \sim \pi(\cdot | S_0)$
 $T \leftarrow \infty$
 Loop for $t = 0, 1, 2, \dots$:
 If $t < T$, then:
 Take action A_t
 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 If S_{t+1} is terminal, then:
 $T \leftarrow t + 1$
 else:
 Select and store an action $A_{t+1} \sim \pi(\cdot | S_{t+1})$
 $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
 If $\tau \geq 0$:
 $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
 $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$
 If π is being learned, then ensure that $\pi(\cdot | S_\tau)$ is ε -greedy wrt Q
 Until $\tau = T - 1$

Off-policy n -step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q , or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:
 Initialize and store $S_0 \neq \text{terminal}$
 Select and store an action $A_0 \sim b(\cdot | S_0)$
 $T \leftarrow \infty$
 Loop for $t = 0, 1, 2, \dots$:
 If $t < T$, then:
 Take action A_t
 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
 If S_{t+1} is terminal, then:
 $T \leftarrow t + 1$
 else:
 Select and store an action $A_{t+1} \sim b(\cdot | S_{t+1})$
 $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
 If $\tau \geq 0$:
 $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i | S_i)}{b(A_i | S_i)}$ ($\rho_{\tau+1:\tau+n}$)
 $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)
 $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \rho [G - Q(S_\tau, A_\tau)]$
 If π is being learned, then ensure that $\pi(\cdot | S_\tau)$ is greedy wrt Q
 Until $\tau = T - 1$



n -step Tree Backup for estimating $Q \approx q_*$ or q_π

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize π to be greedy with respect to Q , or as a fixed given policy

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

All store and access operations can take their index mod $n + 1$

Loop for each episode:

Initialize and store $S_0 \neq \text{terminal}$

Choose an action A_0 arbitrarily as a function of S_0 ; Store A_0

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \dots$:

 If $t < T$:

 Take action A_t ; observe and store the next reward and state as R_{t+1}, S_{t+1}

 If S_{t+1} is terminal:

$T \leftarrow t + 1$

 else:

 Choose an action A_{t+1} arbitrarily as a function of S_{t+1} ; Store A_{t+1}

$\tau \leftarrow t + 1 - n$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

 If $t + 1 \geq T$:

$G \leftarrow R_T$

 else

$G \leftarrow R_{t+1} + \gamma \sum_a \pi(a|S_{t+1})Q(S_{t+1}, a)$

 Loop for $k = \min(t, T - 1)$ down through $\tau + 1$:

$G \leftarrow R_k + \gamma \sum_{a \neq A_k} \pi(a|S_k)Q(S_k, a) + \gamma \pi(A_k|S_k)G$

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$

 If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q

Until $\tau = T - 1$

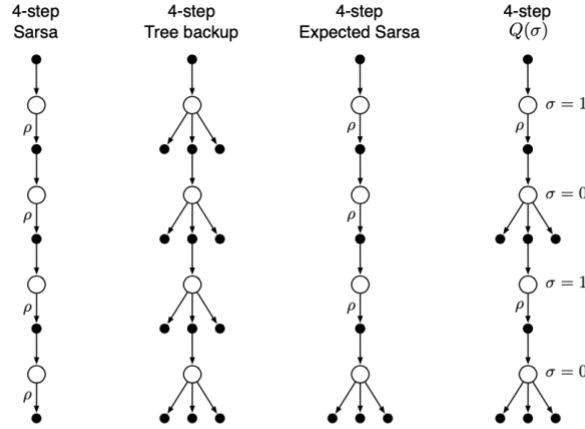


Figure 7.5: The backup diagrams of the three kinds of n -step action-value updates considered so far in this chapter (4-step case) plus the backup diagram of a fourth kind of update that unifies them all. The label ‘ ρ ’ indicates half transitions on which importance sampling is required in the off-policy case. The fourth kind of update unifies all the others by choosing on a state-by-state basis whether to sample ($\sigma_t = 1$) or not ($\sigma_t = 0$).

Off-policy n -step $Q(\sigma)$ for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize π to be greedy with respect to Q , or else it is a fixed given policy

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

All store and access operations can take their index mod $n + 1$

Loop for each episode:

Initialize and store $S_0 \neq \text{terminal}$

Choose and store an action $A_0 \sim b(\cdot|S_0)$

$T \leftarrow \infty$

Loop for $t = 0, 1, 2, \dots$:

 If $t < T$:

 Take action A_t ; observe and store the next reward and state as R_{t+1}, S_{t+1}

 If S_{t+1} is terminal:

$T \leftarrow t + 1$

 else:

 Choose and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$

 Select and store σ_{t+1}

 Store $\frac{\pi(A_{t+1}|S_{t+1})}{b(A_{t+1}|S_{t+1})}$ as ρ_{t+1}

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

 If $t + 1 < T$:

$G \leftarrow Q(S_{t+1}, A_{t+1})$

 Loop for $k = \min(t + 1, T)$ down through $\tau + 1$:

 if $k = T$:

$G \leftarrow R_T$

 else:

$\bar{V} \leftarrow \sum_a \pi(a|S_k) Q(S_k, a)$

$G \leftarrow R_k + \gamma(\sigma_k \rho_k + (1 - \sigma_k) \pi(A_k|S_k))(G - Q(S_k, A_k)) + \gamma \bar{V}$

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$

 If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q

Until $\tau = T - 1$